Personal Network Assistant

Date: 7 July 2026 Team ID: Not provided Project Name: Personal Network Assistant

1. Project Title

Personal Network Assistant is a web application for preparing professional networking conversations.

2. Project Description

The application helps users prepare for networking interactions through saved profiles, event analysis, personalized conversation starters, tone control, fact checking, conversation history, and feedback. It combines a Streamlit frontend, FastAPI backend, SQLAlchemy persistence, local DistilBERT theme extraction, Google Gemini conversation generation, and Wikipedia fact summaries.

3. Real-world Use Scenarios

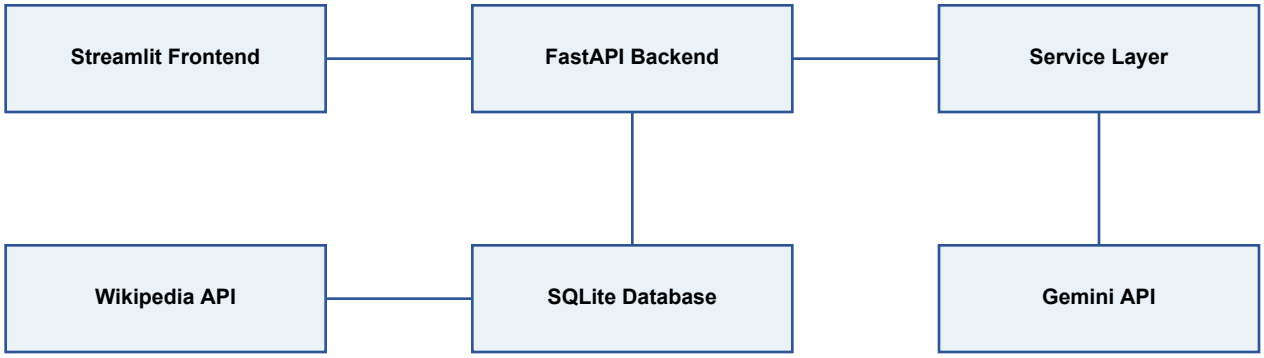
- A student preparing for a career fair saves profile details, enters event context, and receives targeted conversation starters.
- A professional attending an industry conference selects a formal or casual tone and fact-checks topics before speaking with attendees.
- A user reviews conversation history and feedback to understand which suggestions were useful.

4. Technical Architecture

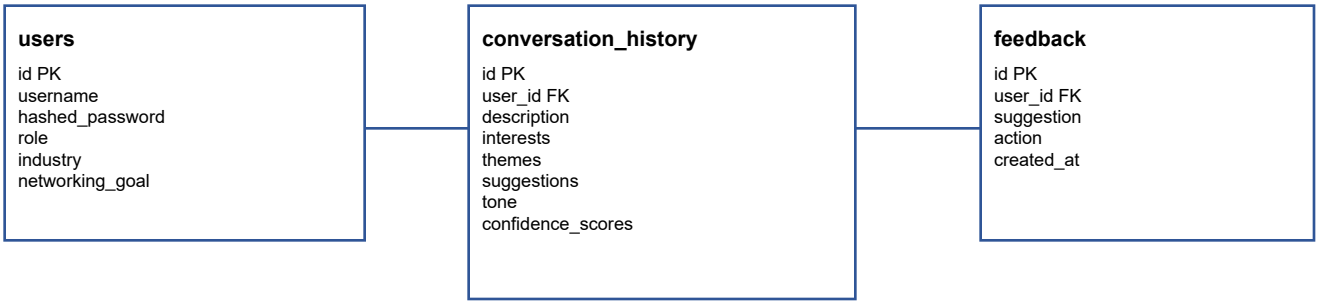
The architecture uses a Streamlit frontend for forms and pages, a FastAPI backend for JSON APIs, a service layer for AI and persistence operations, SQLAlchemy models over SQLite, JWT bearer authentication, and selected rate limiting for expensive endpoints.

Architecture and Data Model

5. Architecture Diagram



6. Database / ER Diagram



Prerequisites and Workflow

7. Prerequisites

- Python 3.10+ with Python 3.11 recommended
- Git, pip, and virtual environment support
- GEMINI_API_KEY for conversation generation
- Docker Desktop optional for Docker Compose

8. Project Workflow

- Architecture and environment setup
- Authentication and persistence
- Profile and networking features
- Conversation generation and fact checking
- History and feedback collection
- Testing, performance testing, Docker, and documentation

9. Development Milestones

Milestone	Implemented Work
1	FastAPI, Streamlit, environment, and Docker base
2	JWT authentication, SQLAlchemy models, SQLite persistence
3	Profile, event analysis, generation, tone, and scoring
4	Fact checking, history, feedback, and analytics page
5	Automated tests, Locust performance evidence, and documentation PDFs

10. Environment Setup

- Copy .env.example to .env.
- Set SECRET_KEY before non-local use.
- Set GEMINI_API_KEY to enable conversation generation.
- DATABASE_URL defaults to sqlite:///./data/app.db.
- BACKEND_URL defaults to http://127.0.0.1:8000 for local frontend use.

Application Features

11. Authentication and User Management

Users register with username and password. Passwords are hashed with bcrypt. Login returns a signed JWT access token used as a Bearer token for protected API routes.

12. Profile Management

Users can save role, industry, and networking goal through GET and PUT /profile. These optional fields personalize generated conversation starters.

13. Networking Event Analysis

POST /analyze-event extracts up to three themes from event descriptions using a locally loaded DistilBERT zero-shot classification pipeline.

14. Personalized Conversation Generation

POST /generate-conversation analyzes event context, calls Gemini for three starters, scores them, and stores successful results in conversation history.

15. Tone Selection and Scoring

The frontend supports casual, formal, and witty tones. The scorer assigns 1-5 confidence scores with explanations.

16. Fact Checking

POST /fact-check uses Wikipedia REST summaries to return concise topic information or a friendly fallback message.

17. Conversation History

GET /history returns the five most recent conversation-generation records for the authenticated user only.

18. Feedback Collection and Analytics

POST /feedback records like/dislike actions, and GET /feedback-history returns recent entries used by the feedback page chart.

Backend, Frontend, and Persistence

19. Backend API Structure

Method	Endpoint	Purpose
GET	/	Health check
POST	/auth/register	Create account
POST	/auth/login	Issue JWT
POST	/analyze-event	Extract event themes
POST	/generate-conversation	Generate, score, and log starters
POST	/fact-check	Wikipedia summary lookup
POST	/feedback	Record rating
GET	/history	Recent user history
GET	/feedback-history	Recent feedback
GET/PUT	/profile	Read or update profile

20. Frontend Structure

The Streamlit frontend contains a Home generation page plus Profile, Fact Check, History, and Feedback pages. Shared helpers in frontend/common.py manage backend URL, auth headers, login checks, network errors, and styling.

21. Database and Persistence

SQLAlchemy defines users, conversation_history, and feedback tables. SQLite is created automatically at data/app.db by init_db() during startup. Docker Compose persists this database in an app-data volume.

22. Error Handling and Rate Limiting

Global FastAPI handlers return friendly messages for runtime, database, and unexpected errors. /generate-conversation is limited to 10 requests per minute and /fact-check to 20 requests per minute per client.

Testing and Deployment

23. Testing

The pytest suite covers authentication, authorization, API routes, service logic, profile behavior, persistence, security, and error handling. Tests use isolated in-memory SQLite fixtures and mocked external calls where appropriate.

24. Performance Testing

Locust evidence shows a limited local run with 1 virtual user, 40 requests, 0 failures, average response time 22.68 ms, median 6 ms, maximum 360 ms, and current RPS shown as 0.4-0.5. This is not a high-concurrency production benchmark.

25. Docker / Local Deployment

Dockerfile.backend, Dockerfile.frontend, and docker-compose.yml support local containerized execution. No verified public deployment URL is present.

26. Running the Application

Use PowerShell to create a virtual environment, install requirements.txt, copy .env.example to .env, start uvicorn app.main:app --reload, and run streamlit run frontend/app.py. Swagger is available at http://127.0.0.1:8000/docs and the frontend at http://localhost:8501.

27. Exploring the Application Pages

- Home: register, login, enter event details, choose tone, and generate starters.
- Profile: save role, industry, and networking goals.
- Fact Check: verify a topic through Wikipedia.
- History: review recent generated conversations.
- Feedback: view recent like/dislike ratings and chart.

Limitations and Future Enhancements

28. Known Limitations

- Conversation generation requires a valid Gemini API key and internet access.
- SQLite is intended for local and small single-instance use.
- No verified public deployment, demo video, password reset, or email verification is present.
- Performance evidence is limited to a local single-user Locust run; CPU and memory were not measured.

29. Future Enhancements

- PostgreSQL migration with migrations and backups.
- Token refresh, stricter CORS, managed secrets, HTTPS, and audit logging.
- Background queues, caching, retries, pagination, and broader load tests.
- Use collected feedback to improve ranking and personalization quality.

30. Conclusion

Personal Network Assistant delivers a complete local web application for networking preparation, combining authentication, profile-aware AI assistance, fact checking, history, feedback, tests, and Docker-based local execution. The current implementation is suitable for demonstration and local use, with clear next steps for production scaling.